

EVALUACIÓN PRIMER PARCIAL: PROYECTO PRÁCTICO Y DEFENSA ORAL

Simulador de CPU (Arquitectura von Neumann / x86 de 8 bits) y Memoria Principal

"El estudiante que diseña y programa un simulador funcional de una arquitectura de computadoras, comprende con absoluta claridad el funcionamiento interno y la lógica del hardware."

1. CONTEXTO ACADÉMICO Y PROYECCIÓN FORMATIVA

En la disciplina de la Ingeniería de Software, el software de alto nivel cobra verdadero sentido cuando se comprende el soporte físico sobre el cual se ejecutan las instrucciones. Este primer examen parcial reta al estudiante a construir un **simulador interactivo y visual** del ciclo completo de instrucción y de la gestión de memoria en una hoja de cálculo (Excel con VBA o Google Sheets con JavaScript / Apps Script).

Evolución Modular hacia el Segundo Parcial: Este proyecto modela el núcleo de procesamiento (ALU, Registros y Unidad de Control) y la Memoria Principal. Para el **Segundo Parcial**, el sistema se ampliará integrando el **Bus del Sistema multiplexado**, controladores de **Entrada/Salida (I/O)**, periféricos interactivos y gestión de **interrupciones**. La arquitectura de código diseñada hoy debe ser modular, desacoplada y escalable.

2. ESPECIFICACIONES TÉCNICAS DEL SIMULADOR

2.1. Entornos Tecnológicos Admitidos

El estudiante desarrollará la solución en una de las dos plataformas oficiales a su elección:

- **Opción A:** Microsoft Excel con macros programadas en **VBA (Visual Basic for Applications)** (formato `.xlsm` o `.xlsb`).
- **Opción B:** Google Sheets con lógica programada en **JavaScript** mediante **Google Apps Script (GAS)**.

2.2. Arquitectura de Memoria (RAM de 8 bits)

- **Direccionamiento:** 256 posiciones continuas de 8 bits, numeradas de `00h` (`0d`) a `FFh` (`255d`).
- **Ancho de palabra:** 8 bits (1 Byte) por celda.
- **Visualización en la Hoja:** Cuadrícula matricial (ej. matriz 16×16) donde cada dirección permita inspeccionar el valor en formato **Hexadecimal**, **Binario** y **Decimal / Mnemónico**.
- **Segmentación Lógica:** Separación visual clara entre el **Segmento de Código** (espacio donde se cargan las instrucciones del programa) y el **Segmento de Datos** (variables, arreglos y memoria de almacenamiento).
- **Operaciones Primitivas:** Subrutinas dedicadas para operaciones de lectura (`Read(address)`) y escritura (`Write(address, value)`).

2.3. Arquitectura del CPU y Registros Visibles

El procesador simulado debe reflejar en la interfaz los siguientes registros y componentes funcionales:

- **PC (Program Counter):** Puntero de 8 bits que almacena la dirección de la siguiente instrucción en memoria.
- **IR (Instruction Register):** Registro que contiene el código de operación (Opcode) y operandos de la instrucción en curso.
- **MAR (Memory Address Register):** Registro conectado a las líneas de dirección para especificar qué celda de memoria se lee o escribe.
- **MDR / MBR (Memory Buffer/Data Register):** Almacena el dato recién transferido desde o hacia la memoria RAM.
- **AX / AC (Acumulador) y BX:** Registros de propósito general de 8 bits para cómputo aritmético y manipulación de datos.
- **Flags (Registro de Estado):** Banderas de 1 bit actualizadas por la ALU:
 - **ZF (Zero Flag):** Se activa (`1`) si el resultado de la última operación fue cero.
 - **CF (Carry Flag):** Se activa (`1`) si ocurrió un desbordamiento o acarreo sin signo.
 - **SF (Sign Flag):** Refleja el bit más significativo (MSB) del resultado (`1` si es negativo en complemento a 2).
- **ALU (Unidad Aritmético-Lógica):** Módulo responsable de procesar operaciones aritméticas (`ADD`, `SUB`, `INC`, `DEC`) y lógicas (`AND`, `OR`, `XOR`, `NOT`, `CMP`).

2.4. Ciclo de Instrucción Completo (Fetch, Decode, Execute, Store)

El simulador debe descomponer la ejecución de cada instrucción en sus cuatro fases del ciclo de reloj:

1. **Fetch (Búsqueda):** La dirección en **PC** se carga en **MAR**; se lee la RAM hacia **MDR**; el dato pasa al **IR** y se incrementa el **PC**.
2. **Decode (Decodificación):** La Unidad de Control interpreta el Opcode en **IR**, identifica el modo de direccionamiento y prepara los operandos.
3. **Execute (Ejecución):** La ALU efectúa la operación aritmética o lógica, o se calcula la bifurcación condicional/incondicional; se actualizan las banderas.
4. **Store / Write-back (Almacenamiento):** El resultado final se guarda en el registro destino (**AX**, **BX**) o en la celda de memoria apuntada (**MDR** → **RAM[MAR]**).

2.5. Dinamismo, Interfaz y Modos de Ejecución

- **Modo Paso a Paso (Step-by-Step):** Permite avanzar una fase o micro-operación a la vez mediante un botón dedicado. Se debe resaltar visualmente con colores qué registro, bus o celda de memoria está activa en dicho instante.
- **Modo Continuo (Run / Play):** Ejecución secuencial automática con velocidad o retardo (delay) ajustable para observar la animación en tiempo real.
- **Controles Esenciales:** Botones accesibles para **STEP**, **RUN**, **PAUSE**, **RESET** (restaura registros y PC a cero) y **LOAD PROGRAM**.
- **Log de Micro-operaciones:** Panel de registro en la hoja que liste cronológicamente las acciones efectuadas (ej. *[Paso 08] FETCH: MAR=0x12, MDR=0x05 → IR=ADD AX, 0x05*).

2.6. Conjunto Mínimo de Instrucciones (ISA Ensamblador) y Programa Demostrativo

El simulador debe reconocer y ejecutar como mínimo:

- **Transferencia de Datos:** **MOV reg, imm**, **MOV reg, reg**, **LOAD reg, [dir]**, **STORE [dir], reg**.
- **Aritmética y Lógica:** **ADD reg, imm/reg**, **SUB reg, imm/reg**, **INC reg**, **DEC reg**, **CMP reg, imm/reg**.
- **Control de Flujo:** **JMP dir**, **JZ dir** (salto si Zero=1), **JNZ dir** (salto si Zero=0), **HLT** (detiene el reloj).

Programa Demostrativo Obligatorio: Debe cargarse en memoria un programa representativo con bucles y bifurcaciones (ej. multiplicación por sumas sucesivas, serie de Fibonacci hasta desbordamiento de 8 bits, factorial de un entero o cuenta regresiva con guardado condicional en memoria).

3. GESTIÓN DE PROYECTO, ENTREGABLES Y CIERRE DE REPOSITORIO

- **Modalidad Única de Entrega:** En la plataforma académica únicamente se debe registrar y enviar:
 1. **Enlace al Repositorio de GitHub:** Código fuente (.xslm o .gas), scripts y documentación técnica en Markdown.
 2. **Enlace al Tablero de GitHub Projects:** Gestión de tareas y desarrollo bajo metodología Kanban.
(No se deben subir archivos comprimidos .zip ni libros de cálculo directamente; la evaluación se efectúa sobre los repositorios oficiales).
- **Fecha y Hora Límite Improrrogable: Domingo 29 de septiembre a las 23:59 hrs.**
- **Auditoría Estricta del Log de GitHub:** Se auditará el historial cronológico de eventos en GitHub. **No debe existir ningún movimiento, commit, pull request ni cambio de estado de tarjetas en el tablero posterior al 29 de septiembre a las 23:59 hrs.** Cualquier actividad registrada con fecha/hora posterior anulará la entrega.
- **Tablero Kanban (GitHub Projects):** Tablero activo con columnas: *Backlog*, *To Do*, *In Progress*, *In Review / Testing*, *Done*. Cada tarea/issue debe detallar criterios de aceptación (matriz de memoria, decodificador de instrucciones, animación gráfica, etc.).
- **Versionado con Git / GitHub:** Historial de commits continuo y semántico (**feat:**, **fix:**, **docs:**, **refactor:**). Queda penalizado subir el proyecto en un único commit masivo de último momento.
- **Documentación Técnica (`README.md`):** Redactada profesionalmente en Markdown. Debe incluir diagrama de bloques en **Mermaid**, tabla formal de la ISA (códigos de operación, bytes ocupados, descripción), manual de usuario paso a paso y análisis del programa de prueba con traza de registros.

4. DINÁMICA DE LA DEFENSA ORAL (15 MINUTOS ERICTOS)

Minutos 0:00 a 10:00 — Exposición y Demostración

- **Min 0 - 2:** Explicación de la arquitectura diseñada (registros, mapa de memoria y set de instrucciones).
- **Min 2 - 4:** Evidencia de gestión en GitHub Projects (tablero Kanban, issues, commits y README técnico).
- **Min 4 - 10:** Demostración en vivo del simulador: ejecución paso a paso (Fetch → Decode → Execute → Store), saltos condicionales y cambios dinámicos en memoria y banderas.

Minutos 10:00 a 15:00 — Ronda de Preguntas y Código

- Preguntas del docente sobre líneas específicas del código (VBA / Apps Script).
- Modificación o inserción en vivo de una instrucción en memoria para corroborar el dominio conceptual del estudiante.
- Fundamentos teóricos de arquitectura de computadoras aplicados al simulador.

5. RÚBRICA ANALÍTICA DE EVALUACIÓN (PONDERACIÓN: 100 PUNTOS)

La evaluación del proyecto y la defensa oral se rige bajo la siguiente matriz de desempeño, distribuida en cuatro niveles de logro:

Criterio de Evaluación	Pts	Malo / Aplazado (0% - 50%)	Regular / Pasa Raspando (51% - 69%)	Bueno (70% - 89%)	Excelente (90% - 100%)
1. Funcionalidad del Simulador y Ciclo CPU Ejecución de Fetch, Decode, Execute, Store; modos Paso a Paso y Continuo; paradas y saltos.	30	0 - 15 pts: El simulador no ejecuta programas o se traba. No descompone el ciclo en sus 4 fases reales; salta directo al cálculo final sin procesar el ciclo de reloj.	16 - 20 pts: Simula el ciclo pero omite fases (ej. no utiliza MAR/MDR). Funciona con instrucciones simples pero falla en saltos o paradas HLT.	21 - 26 pts: Ciclo completo de 4 fases operativo. Ejecuta instrucciones aritméticas y bifurcaciones condicionales con fallas leves en casos borde de banderas.	27 - 30 pts: Simulación fidedigna, rigurosa y sin fallos del ciclo completo. Ejecución paso a paso y continua impecables, con control de reloj y parada precisa.
2. Arquitectura de Memoria y Registros Mapeo 00h-FFh (256 bytes), PC, IR, MAR, MDR, AX, BX, Banderas (ZF, CF, SF) y ALU.	15	0 - 7 pts: Memoria desordenada, no indexada de 00h a FFh. Registros ausentes o estáticos (el PC no incrementa, no existen IR ni banderas).	8 - 10 pts: Memoria de 256 bytes implementada pero rígida. Faltan registros esenciales (ej. MAR o MDR). Las banderas ZF/CF/SF no se actualizan con fidelidad.	11 - 13 pts: Espacio 00h-FFh bien mapeado (hex/binario). Registros PC, IR, MAR, MDR, AX y banderas presentes y funcionales en la mayoría de instrucciones.	14 - 15 pts: Mapeo 00h-FFh perfecto con clara segmentación código/datos. Registros internos completos y banderas (ZF, CF, SF) sincronizadas al 100% con la ALU.
3. Interfaz Gráfica, Usabilidad y Dinamismo Diseño visual, dinamismo, resaltado de flujo en tiempo real, log de micro-operaciones y ergonomía.	15	0 - 7 pts: Hoja sin diseño, caótica o incomprensible. No hay animación visual ni resaltado del flujo. No se distingue qué componente está activo.	8 - 10 pts: Interfaz elemental. Dispone de botones básicos pero carece de dinamismo (no se ilumina la fase activa ni cuenta con registro de micro-operaciones).	11 - 13 pts: Interfaz limpia y bien organizada. Dispone de controles Step/Run/Reset y resaltado visual de registros y celdas de memoria en ejecución.	14 - 15 pts: Diseño gráfico excepcional, didáctico y profesional. Animación del flujo en tiempo real, log cronológico detallado y controles ergonómicos.
4. Metodología Kanban y Control de Versiones Tablero en GitHub Projects, desglose de tareas, commits atómicos continuos a lo largo del desarrollo.	15	0 - 7 pts: No existe tablero Kanban o está vacío. Repositorio con un solo commit ("entrega final") sin evidencia de proceso de desarrollo.	8 - 10 pts: Tablero Kanban con pocas tarjetas generales desactualizadas. Historial de Git con pocos commits y mensajes vagos ("cambios", "subiendo").	11 - 13 pts: Tablero Kanban con historias e issues organizados en columnas. Flujo continuo de commits atómicos con mensajes claros a lo largo de las semanas.	14 - 15 pts: Gestión ágil ejemplar en GitHub Projects (tarjetas desglosadas con criterios de aceptación). Historial de Git limpio, semántico, continuo y estructurado.
5. Documentación Técnica en Markdown README.md con diagrama Mermaid, tabla ISA de instrucciones, manual de usuario y análisis del programa.	10	0 - 5 pts: Documento ausente, copiado o de escasas líneas. Sin explicaciones técnicas, sin tabla de opcodes ni guía de usuario.	6 - 7 pts: Documentación básica en texto plano. Carece de diagramas de arquitectura, tablas de opcodes o análisis paso a paso del código ensamblador.	8 - 9 pts: Buen README.md con formato ordenado, tabla del repertorio de instrucciones, guía de ejecución y explicación del programa de prueba.	10 pts: Documentación de estándar profesional: diagrama de arquitectura en Mermaid, tabla exhaustiva de la ISA, manual de usuario y traza matemática completa.
6. Defensa Oral y Dominio Conceptual Puntualidad en los 10+5 min, fluidez, solvencia ante modificación de código/memoria en vivo.	15	0 - 7 pts: No se ajusta al tiempo. Incapaz de explicar el funcionamiento de su propio código o conceptos básicos de CPU. Evidencia de copia o plagio.	8 - 10 pts: Exposición dubitativa. Responde parcialmente a las preguntas técnicas con dudas notorias sobre la relación entre el hardware y el código.	11 - 13 pts: Buena presentación y respeto del tiempo. Demuestra en vivo el funcionamiento y responde con seguridad a la mayoría de las preguntas teóricas.	14 - 15 pts: Defensa impecable, precisa y elocuente. Demuestra dominio absoluto al modificar código o memoria en vivo y responder preguntas teóricas avanzadas.

6. ESCALA CUALITATIVA DE CALIFICACIÓN FINAL

- **90 a 100 Puntos — EXCELENTE:** Proyecto de nivel profesional. El estudiante demuestra dominio total de la arquitectura de computadoras y queda plenamente habilitado para abordar con éxito el Parcial 2 (Buses e I/O).
- **70 a 89 Puntos — BUENO:** Aprobado con solvencia técnica. Cumple las metas pedagógicas con observaciones menores de diseño o documentación.
- **51 a 69 Puntos — REGULAR (PASA RASPANDO):** Cumplimiento mínimo aceptable. Se evidencian debilidades conceptuales en el ciclo de reloj o desorden en el código y metodología.
- **0 a 50 Puntos — REPROBADO / APLAZADO:** No alcanza los estándares mínimos de la asignatura ni demuestra comprensión de la arquitectura.

Plazo de Entrega y Cierre de Actividad: Fecha límite máxima: **Domingo 29 de septiembre a las 23:59 hrs** mediante el envío exclusivo de los dos enlaces oficiales (GitHub Repository y GitHub Project). Todo commit o movimiento en el log de GitHub registrado después de dicha hora no será admitido y causará la anulación del criterio de entrega.

Integridad y Defensa: La defensa oral individual de 15 minutos es de carácter obligatorio. Todo indicio de código duplicado o plagio no justificado será sancionado con calificación de **0 (Cero)** y remisión reglamentaria institucional.